

Report Draft

1. Output Results of ADD

Iteration 1: Establishing an Overall System Structure

Iteration 1: Establishing an Overall System Structure

ADD Step 1: Review Inputs

Architectural Drivers for This Iteration

- Use Cases: HPS-1 (Log In), HPS-2 (Change Prices), HPS-3 (Query Prices), HPS-4 (Manage Hotels), HPS-5 (Manage Rates), HPS-6 (Manage Users)
- Quality Attributes:
 - QA-1 (Performance): High importance, high difficulty.
 - QA-2 (Reliability): High importance, high difficulty.
 - QA-3 (Availability): High importance, high difficulty.
 - QA-4 (Scalability): High importance, high difficulty.
 - QA-5 (Security): High importance, medium difficulty.
- Architectural Concerns:
 - CRN-1 (Establish an overall initial system structure).
 - CRN-2 (Leverage the team's knowledge about Java technologies, Angular framework, and Kafka).
 - CRN-6 (Favor a cloud-native approach).
- Constraints:
 - CON-1 (Web browser access on multiple platforms and devices).
 - CON-2 (User management via cloud identity service; resources hosted in the cloud).
 - CON-5 (REST APIs initially, with potential for other protocols).
 - CON-6 (Cloud-native design).

ADD Step 2: Establish the Iteration Goal by Selecting Drivers

Iteration Goal

To establish the overall system structure that supports the primary functionality of the Hotel Pricing System while addressing key quality attributes such as performance, reliability, availability, scalability, and security. The goal is to define a cloud-native architecture using known technologies (Java, Angular, Kafka) and ensure compatibility with REST APIs and future protocol extensions.

ADD Step 3: Choose One or More Elements of the System to Refine

Element Chosen for Refinement

The entire system is selected for refinement since this is a greenfield project. We will decompose the system into major architectural components that align with the functional and non-functional requirements.

ADD Step 4: Choose One or More Design Concepts That Satisfy the Selected Drivers

Design Concepts Considered

1. Monolithic Architecture: Simple to develop but lacks scalability and modifiability.
2. Microservices Architecture: Supports scalability, modifiability, and separation of concerns, but increases complexity.
3. Event-Driven Architecture: Enables asynchronous communication and decoupling of services, which supports reliability and scalability.
4. Cloud-Native Architecture: Aligns with constraints and allows for elasticity, scalability, and deployment flexibility.

Selected Design Concept

A cloud-native microservices architecture with event-driven capabilities is chosen. It satisfies the following drivers:

- QA-1 (Performance): Microservices allow for horizontal scaling and efficient resource allocation.
- QA-2 (Reliability): Event-driven patterns support fault tolerance and message durability.
- QA-3 (Availability): Cloud-native deployment ensures high availability through redundancy and failover mechanisms.
- QA-4 (Scalability): Microservices can scale independently based on demand.
- QA-5 (Security): Centralized authentication and authorization are supported via cloud identity services.
- CRN-1 (System structure): Provides a modular and extensible structure.
- CRN-2 (Team expertise): Leverages Java, Angular, and Kafka.
- CRN-6 (Cloud-native): Fully aligned with constraint CON-6.

ADD Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

Major Components

1. Frontend Application (Angular):
 - Responsibility: Provide user interface for hotel pricing management.
 - Interface: Communicates with backend via REST APIs.
 - Constraint Support: CON-1 (cross-platform web access).
2. Authentication Service:
 - Responsibility: Validate user credentials against cloud identity service.
 - Interface: REST API for login and authorization.
 - Constraint Support: CON-2 (user management via cloud identity service).
3. Hotel Management Service:
 - Responsibility: Manage hotel data including tax rates, room types, and available rates.
 - Interface: REST API for CRUD operations.
 - Use Case Support: HPS-4 (Manage Hotels).
4. Rate Management Service:
 - Responsibility: Manage rate definitions and calculation rules.
 - Interface: REST API for rate configuration.
 - Use Case Support: HPS-5 (Manage Rates).
5. Price Engine Service:

- Responsibility: Calculate prices based on base rates and fixed rates.
- Interface: REST API for price simulation and actual updates.
- Use Case Support: HPS-2 (Change Prices), HPS-3 (Query Prices).

6. Channel Management Integration Service:

- Responsibility: Push updated prices to external Channel Management Systems.
- Interface: REST API for integration with external systems.
- Use Case Support: HPS-2 (Change Prices).

7. Kafka Message Broker:

- Responsibility: Facilitate asynchronous communication between services.
- Interface: Publish/subscribe model for events like price changes.
- Quality Attribute Support: QA-2 (Reliability), QA-4 (Scalability).

8. Monitoring and Logging Service:

- Responsibility: Collect metrics and logs for monitoring and debugging.
- Interface: Exposes dashboards and APIs for observability.
- Quality Attribute Support: QA-8 (Monitorability), QA-9 (Testability).

9. API Gateway:

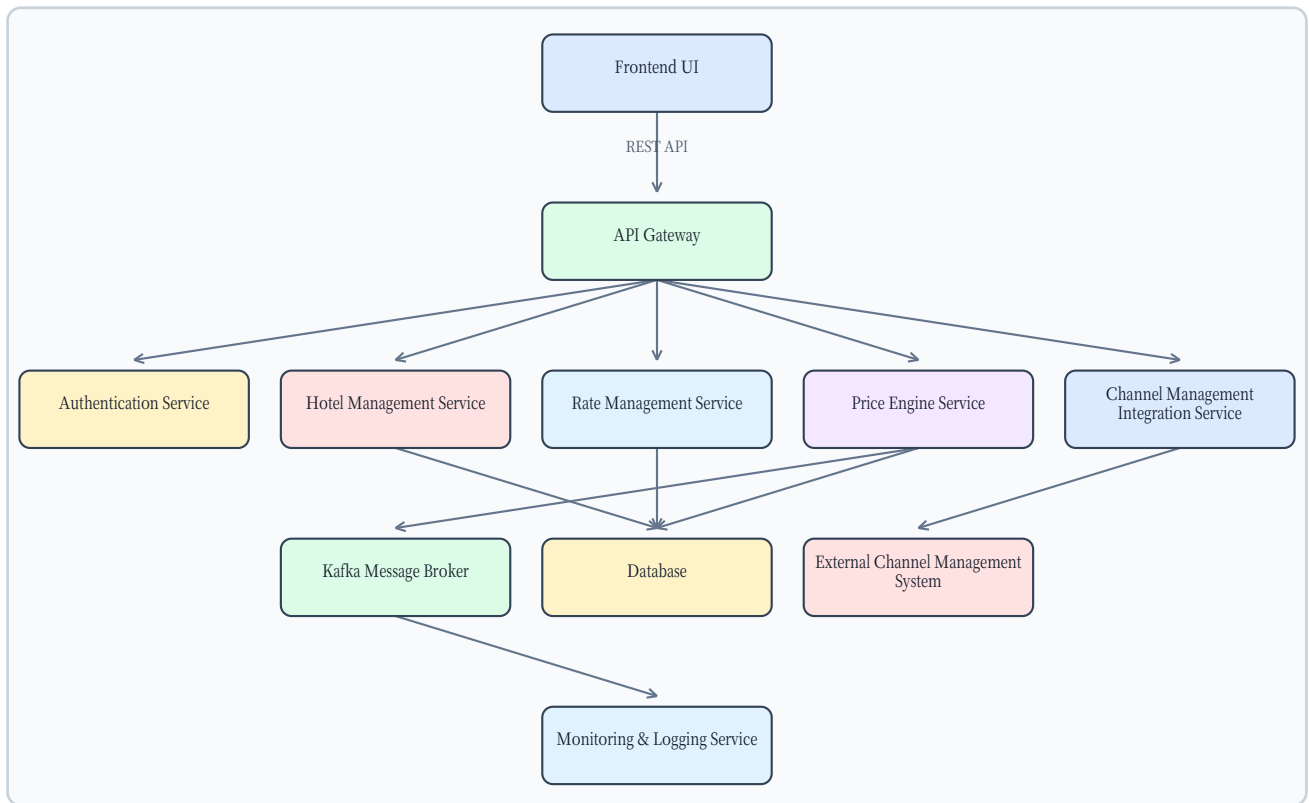
- Responsibility: Route requests to appropriate services, enforce security policies, and manage rate limiting.
- Interface: Single entry point for all client requests.
- Quality Attribute Support: QA-5 (Security), QA-6 (Modifiability).

10. Database Layer:

- Responsibility: Store persistent data such as hotel information, rate configurations, and user permissions.
- Interface: SQL or NoSQL database accessed via ORM or direct queries.
- Quality Attribute Support: QA-2 (Reliability), QA-3 (Availability).

ADD Step 6: Sketch Views and Record Design Decisions

Mermaid Diagram: High-Level System Architecture



Significant Design Decisions and Rationale

Decision	Rationale	Traceability
Adopting a cloud-native microservices architecture	Supports scalability, reliability, and alignment with team expertise and constraints.	CRN-1, CRN-2, CRN-6, QA-1, QA-2, QA-3, QA-4
Using Kafka for event-driven communication	Ensures asynchronous processing and fault tolerance for price updates.	QA-2, QA-4
Implementing an API Gateway	Centralizes security enforcement and routing logic.	QA-5, QA-6
Separating frontend and backend	Enables independent development and deployment.	CON-1, QA-5
Using a centralized database layer	Ensures data consistency and persistence across services.	QA-2, QA-3

ADD Step 7: Perform Analysis of Current Design and Review Iteration Goal and Achievement of Design Purpose

Evaluation of Iteration Goal

The iteration goal was to establish the overall system structure that supports the primary functionality and addresses key quality attributes. The current design provides a clear decomposition of the system into modular components, each with well-defined responsibilities and interfaces. The use of a cloud-native microservices architecture with event-driven communication supports scalability, reliability, and performance requirements.

Next Steps

The next iteration will focus on identifying structures to support the primary functionality in more detail, particularly around the Price Engine Service and its interactions with other components.

STATUS: PASS

Findings: All required ADD steps were completed for Iteration 1. The system structure was established using a cloud-native microservices architecture with event-driven communication. The design decisions are traceable to the provided use cases, quality

attributes, architectural concerns, and constraints. A Mermaid diagram was included to visualize the architecture.

Iteration 2: Identifying Structures to Support Primary Functionality

Iteration 2: Identifying Structures to Support Primary Functionality

ADD Step 2: Establish the Iteration Goal by Selecting Drivers

Iteration Goal

To identify and refine architectural structures that directly support the primary functionality of the Hotel Pricing System, particularly focusing on:

- HPS-2 (Change Prices): Enabling efficient price simulation and publication with performance and reliability.
- HPS-3 (Query Prices): Supporting fast and scalable query operations for internal users and external systems.
- QA-1 (Performance): Ensuring price changes are processed and published within 100 ms.
- QA-4 (Scalability): Supporting up to 1,000,000 queries per day without significant latency increase.
- CRN-2 (Team expertise in Java, Angular, Kafka): Leveraging known technologies for implementation.

This iteration will focus on refining the Price Engine Service and its interactions with other components to ensure it meets functional and non-functional requirements.

ADD Step 3: Choose One or More Elements of the System to Refine

Element Chosen for Refinement

The Price Engine Service is selected for refinement. It is central to the system's core functionality (price calculation, simulation, and publishing) and must meet strict performance and scalability requirements.

ADD Step 4: Choose One or More Design Concepts That Satisfy the Selected Drivers

Design Concepts Considered

1. In-Memory Caching: To reduce database load and improve query response times.
2. Asynchronous Processing: For decoupling price change requests from immediate execution.
3. Batch Processing: For aggregating multiple price updates to optimize resource usage.
4. CQRS (Command Query Responsibility Segregation): Separating read and write models to optimize performance and scalability.
5. Rate Limiting and Throttling: To manage API traffic and prevent overloading.
6. Event Sourcing: To maintain a history of all price changes for audit and traceability.

Selected Design Concept

A combination of CQRS, in-memory caching, and asynchronous event processing via Kafka is selected. This approach supports:

- QA-1 (Performance): CQRS enables optimized read/write paths; caching reduces latency.
- QA-4 (Scalability): Read and write models can scale independently.
- QA-2 (Reliability): Event sourcing ensures durability and traceability.
- CRN-2 (Java/Kafka): Leverages team knowledge and existing infrastructure.

ADD Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

Refined Components

1. Price Engine Service (Refined)

- Responsibilities:
 - Accepts price change commands (e.g., base rate update).
 - Simulates price changes using business rules.
 - Publishes actual price changes to Channel Management System.
 - Maintains an event log for audit and replay.
- Interfaces:
 - REST API for command ingestion (e.g., `/api/pricing/update`).
 - Kafka topic for publishing events (`price-changes-topic`).
 - Internal message queue for asynchronous processing.
- Quality Attribute Support:
 - QA-1 (Performance): In-memory caching and CQRS.
 - QA-2 (Reliability): Event sourcing and Kafka durability.
 - QA-4 (Scalability): Independent scaling of read/write models.

2. Price Query Service

- Responsibilities:
 - Provides fast access to current pricing data.
 - Supports filtering by hotel, date, room type, etc.
- Interfaces:
 - REST API for querying prices (e.g., `/api/pricing/query`).
 - GraphQL endpoint for flexible querying (future extensibility).
- Quality Attribute Support:
 - QA-4 (Scalability): Can be scaled independently.
 - QA-1 (Performance): Uses in-memory cache backed by database.

3. Business Rule Engine

- Responsibilities:
 - Applies rate calculation logic (e.g., tax, discounts).
 - Validates price constraints (e.g., minimum/maximum rates).
- Interfaces:
 - Internal API for rule evaluation.
 - Configurable rule definitions stored in database.
- Quality Attribute Support:
 - QA-1 (Performance): Rules are precompiled and cached.
 - QA-9 (Testability): Rules can be unit tested independently.

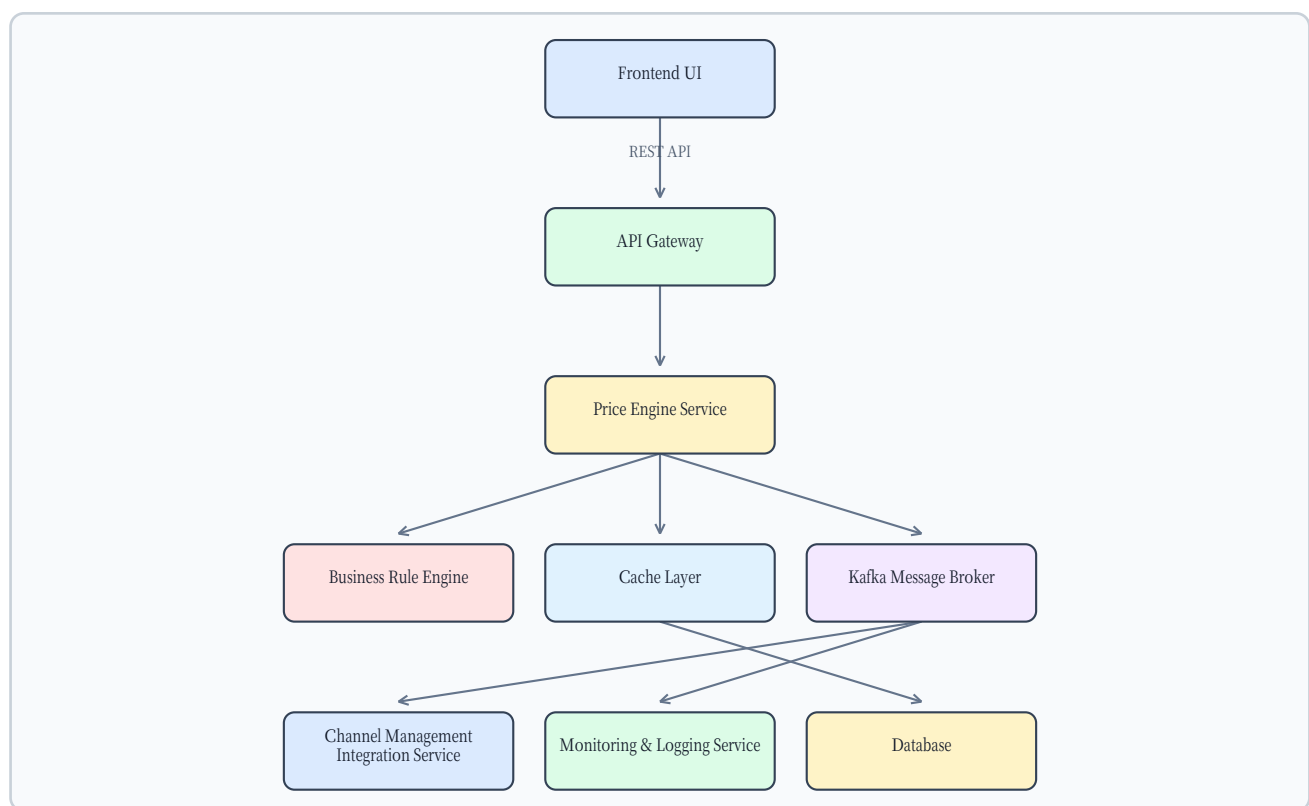
4. Cache Layer

- Responsibilities:

- Stores frequently accessed pricing data in memory.
- Invalidates cache entries when price changes occur.
- Interfaces:
 - Internal API for cache reads/writes.
 - Listens to Kafka events for invalidation.
- Quality Attribute Support:
 - QA-1 (Performance): Reduces database roundtrips.
 - QA-4 (Scalability): Distributed cache (e.g., Redis).

ADD Step 6: Sketch Views and Record Design Decisions

Mermaid Diagram: Price Engine Architecture



Significant Design Decisions and Rationale

Decision	Rationale	Traceability
Implementing CQRS for Price Engine	Separates read and write models to optimize performance and scalability.	QA-1, QA-4
Using in-memory caching	Reduces latency for frequent price queries.	QA-1, QA-4
Asynchronous event processing via Kafka	Ensures reliable delivery of price changes and supports decoupling.	QA-2, QA-4
Adding a Business Rule Engine	Centralizes and modularizes rate calculation logic.	HPS-2, QA-1
Introducing a dedicated Price Query Service	Enables independent scaling and faster query responses.	QA-4, QA-1

ADD Step 7: Perform Analysis of Current Design and Review Iteration Goal and Achievement of Design Purpose

Evaluation of Iteration Goal

The iteration goal was to identify and refine structures supporting the primary functionality, especially around price management and querying. The refined design introduces key components such as the Price Engine Service, Business Rule Engine, and Cache Layer, which directly address the use cases HPS-2 and HPS-3 while meeting quality attributes like performance, scalability, and reliability.

Next Steps

The next iteration will focus on addressing reliability and availability concerns, including fault tolerance, redundancy, and monitoring strategies.

STATUS: PASS

Findings: All required ADD steps were completed for Iteration 2. The Price Engine Service and related components were refined to support primary functionality and meet critical quality attributes. Design decisions are traceable to use cases, quality attributes, and constraints. A Mermaid diagram was included to visualize the architecture.

Iteration 3: Addressing Reliability and Availability Quality Attributes

Iteration 3: Addressing Reliability and Availability Quality Attributes

ADD Step 2: Establish the Iteration Goal by Selecting Drivers

Iteration Goal

To address reliability (QA-2) and availability (QA-3) quality attributes by designing fault-tolerant, highly available structures that ensure:

- 100% successful price change publication and delivery to Channel Management System.
- Pricing query uptime of 99.9% outside maintenance windows.
- Resilience against failures in critical components like the Price Engine Service, Kafka, and external integrations.

This iteration will focus on refining redundancy strategies, failure recovery mechanisms, and monitoring capabilities for key services.

ADD Step 3: Choose One or More Elements of the System to Refine

Element Chosen for Refinement

The following elements are selected for refinement:

1. Price Engine Service
2. Kafka Message Broker
3. Channel Management Integration Service
4. Monitoring & Logging Service

These components are central to ensuring reliability and availability across the system.

ADD Step 4: Choose One or More Design Concepts That Satisfy the Selected Drivers

Design Concepts Considered

1. Redundancy and Failover: Deploy multiple instances of critical services with automatic failover.
2. Message Durability and Retries: Ensure message persistence and retry logic for failed deliveries.
3. Health Checks and Self-Healing: Implement health checks and auto-restart capabilities for services.
4. Circuit Breaker Pattern: Prevent cascading failures by isolating failing components.
5. Distributed Tracing and Monitoring: Enable end-to-end visibility into service interactions.
6. Database Replication and Read Replicas: Improve database availability and reduce read latency.

Selected Design Concept

A combination of redundancy, message durability, health checks, and distributed tracing is selected to satisfy QA-2 (Reliability) and QA-3 (Availability). These concepts align with cloud-native best practices and support the team's familiarity with Java and Kafka.

ADD Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

Refined Components

1. Price Engine Service (Enhanced)

- Responsibilities:
 - Run in a clustered mode with multiple replicas.
 - Use circuit breakers to isolate failures during price simulation or publishing.
 - Support graceful degradation when external systems (e.g., Channel Management) are unavailable.
- Interfaces:
 - REST API with built-in rate limiting and timeout controls.
 - Health check endpoint (`/health`) for load balancers.
- Quality Attribute Support:
 - QA-2 (Reliability): Circuit breakers prevent cascading failures.
 - QA-3 (Availability): Clustered deployment ensures high availability.

2. Kafka Message Broker (Enhanced)

- Responsibilities:
 - Configure topics with replication factor > 1 .
 - Enable message retention policies and dead-letter queues for failed messages.
- Interfaces:
 - Producer/consumer APIs with acknowledgment and retry logic.
 - Monitoring dashboard for topic health and consumer lag.
- Quality Attribute Support:
 - QA-2 (Reliability): Message durability and retries.
 - QA-3 (Availability): Redundant brokers prevent single points of failure.

3. Channel Management Integration Service (Enhanced)

- Responsibilities:
 - Retry failed integrations using exponential backoff.

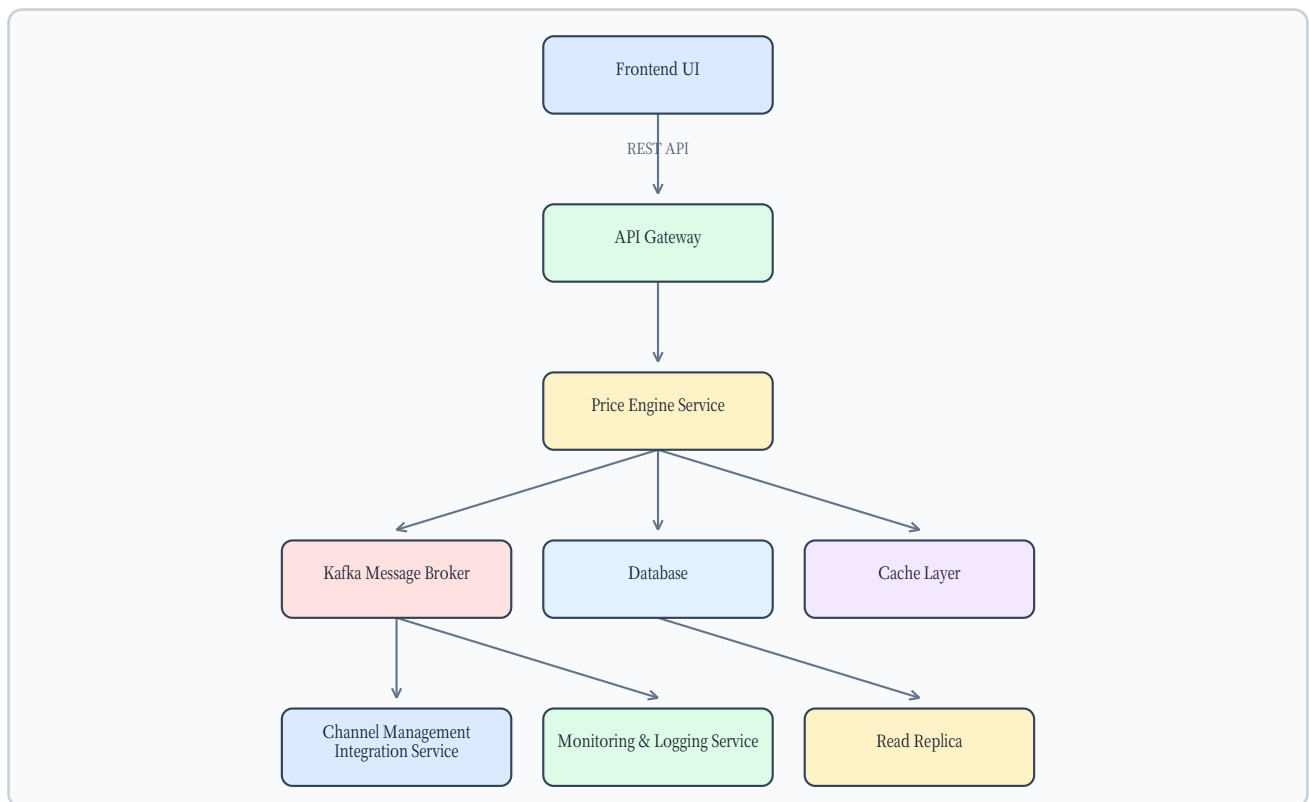
- Store pending updates in a durable queue until external system becomes available.
- Interfaces:
 - REST API for integration with external systems.
 - Internal queue for pending updates.
- Quality Attribute Support:
 - QA-2 (Reliability): Ensures all price changes are eventually delivered.
 - QA-3 (Availability): Graceful handling of external system outages.

4. Monitoring & Logging Service (Enhanced)

- Responsibilities:
 - Collect logs from all services and provide centralized dashboards.
 - Monitor service health and alert on failures.
 - Track metrics such as request latency, success rates, and error counts.
- Interfaces:
 - Exposes Prometheus metrics endpoints.
 - Provides Grafana dashboards for real-time monitoring.
- Quality Attribute Support:
 - QA-8 (Monitorability): Full observability of system behavior.
 - QA-3 (Availability): Early detection of potential outages.

ADD Step 6: Sketch Views and Record Design Decisions

Mermaid Diagram: Reliability and Availability Architecture



Significant Design Decisions and Rationale

Decision	Rationale	Traceability
Clustered deployment of Price Engine Service	Ensures high availability and fault tolerance.	QA-3, QA-2
Kafka topic replication and dead-letter queues	Prevents data loss and enables message recovery.	QA-2, QA-3
Exponential backoff and retry logic in Channel Management Integration Service	Guarantees eventual delivery of price changes.	QA-2
Database read replicas	Improves query performance and availability.	QA-3
Centralized monitoring and distributed tracing	Enables early detection and resolution of issues.	QA-8, QA-3

ADD Step 7: Perform Analysis of Current Design and Review Iteration Goal and Achievement of Design Purpose

Evaluation of Iteration Goal

The iteration goal was to address reliability and availability concerns through architectural refinements. The current design introduces clustered deployments, message durability, retry logic, and enhanced monitoring. These changes directly support the requirements for 100% reliable price change publication and 99.9% uptime for pricing queries.

Next Steps

The next iteration will focus on addressing development and operations concerns, including continuous deployment, testability, and modifiability.

STATUS: PASS

Findings: All required ADD steps were completed for Iteration 3. The architecture now includes reliability and availability features such as clustering, message durability, retry logic, and monitoring. Design decisions are traceable to use cases, quality attributes, and constraints. A Mermaid diagram was included to visualize the architecture.

Iteration 4: Addressing Development and Operations

Iteration 4: Addressing Development and Operations

ADD Step 2: Establish the Iteration Goal by Selecting Drivers

Iteration Goal

To address development and operations concerns, specifically:

- CRN-3 (Allocate work to development team members): Enable modular development and independent deployment of services.
- CRN-4 (Avoid technical debt): Ensure clean separation of concerns and maintainable code structure.
- CRN-5 (Set up continuous deployment infrastructure): Support automated testing, building, and deployment pipelines.
- QA-6 (Modifiability): Allow for protocol extensions without affecting core components.
- QA-7 (Deployability): Enable seamless movement between environments without code changes.
- QA-9 (Testability): Support integration testing with external systems isolated.

This iteration will focus on refining the CI/CD pipeline, modularization strategy, and testability mechanisms.

ADD Step 3: Choose One or More Elements of the System to Refine

Element Chosen for Refinement

The following elements are selected for refinement:

1. CI/CD Pipeline
2. Service Modularization Strategy
3. Testing Infrastructure
4. Environment Configuration Management

These elements directly support development and operations goals such as deployability, testability, and modifiability.

ADD Step 4: Choose One or More Design Concepts That Satisfy the Selected Drivers

Design Concepts Considered

1. Infrastructure as Code (IaC): Define cloud resources using declarative configuration files.
2. Containerization: Use Docker and Kubernetes for consistent deployments across environments.
3. Modular Monorepo: Organize microservices in a single repository with shared libraries.
4. Automated Testing Framework: Implement unit, integration, and end-to-end tests.
5. Feature Flags: Enable safe rollouts and A/B testing.
6. Multi-Stage Builds: Separate build, test, and deployment stages in CI/CD pipelines.

Selected Design Concept

A combination of containerization, modular monorepo, automated testing, and multi-stage CI/CD pipelines is selected. This approach supports:

- CRN-3 (Team allocation): Independent service development and deployment.
- CRN-4 (Technical debt avoidance): Clean separation of concerns and reusable components.
- CRN-5 (Continuous deployment): Automated pipelines for fast feedback and delivery.
- QA-6 (Modifiability): Protocol extensions can be added via new endpoints without core changes.
- QA-7 (Deployability): Environment-specific configurations managed through IaC.
- QA-9 (Testability): Comprehensive testing at all levels.

ADD Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

Refined Components

1. CI/CD Pipeline

- Responsibilities:
 - Automate build, test, and deployment processes.
 - Enforce quality gates before deployment.
 - Integrate with Git-based platform for version control.
- Interfaces:
 - Webhooks from Git platform to trigger pipeline.
 - Integration with container registry (e.g., Docker Hub).
 - Deployment scripts for Kubernetes clusters.
- Quality Attribute Support:

- QA-7 (Deployability): No code changes needed when moving between environments.
- CRN-5 (Continuous deployment): Full automation of release process.

2. Service Modularization Strategy

- Responsibilities:
 - Each microservice is developed and deployed independently.
 - Shared libraries are versioned and published to internal artifact repositories.
 - Services communicate via well-defined APIs and events.
- Interfaces:
 - REST/gRPC APIs for inter-service communication.
 - Internal artifact repository for shared libraries.
- Quality Attribute Support:
 - QA-6 (Modifiability): New protocols can be added without changing existing services.
 - CRN-3 (Team allocation): Teams can own and evolve specific services.

3. Testing Infrastructure

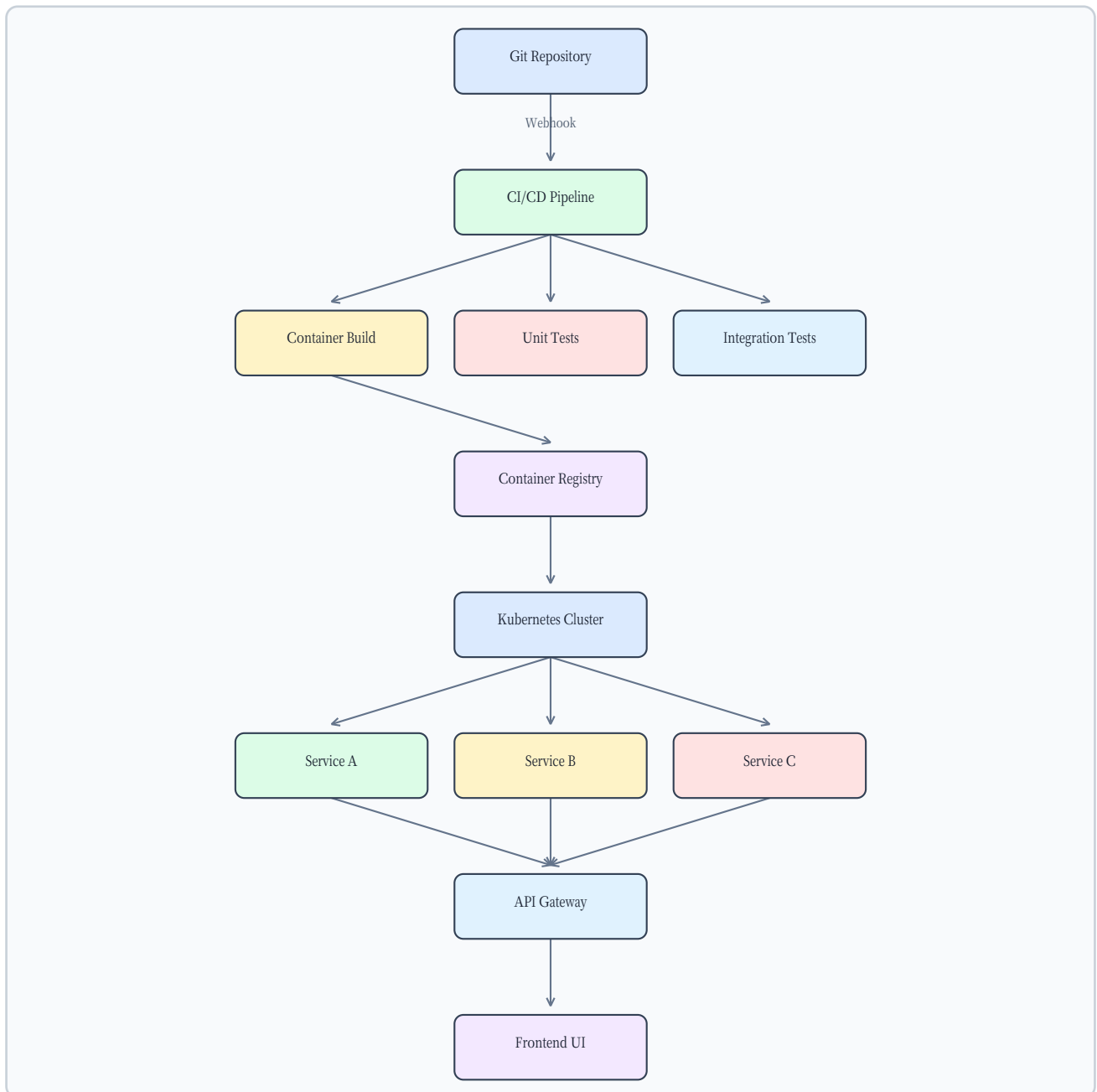
- Responsibilities:
 - Provide mock services for external dependencies during testing.
 - Run unit, integration, and end-to-end tests automatically.
 - Generate test coverage reports and enforce thresholds.
- Interfaces:
 - Test frameworks (e.g., JUnit, Mockito).
 - Mock server for simulating Channel Management System.
 - Integration test suite for API validation.
- Quality Attribute Support:
 - QA-9 (Testability): 100% test coverage with isolation from external systems.
 - CRN-4 (Avoid technical debt): Early detection of issues through testing.

4. Environment Configuration Management

- Responsibilities:
 - Store environment-specific settings (e.g., credentials, URLs) in secure configuration stores.
 - Inject configuration into services at runtime.
 - Support multiple environments (dev, staging, prod).
- Interfaces:
 - Configuration management system (e.g., AWS Secrets Manager).
 - Environment variables injected via Kubernetes ConfigMaps.
- Quality Attribute Support:
 - QA-7 (Deployability): No code changes required for environment transitions.
 - CRN-5 (Continuous deployment): Consistent behavior across environments.

ADD Step 6: Sketch Views and Record Design Decisions

Mermaid Diagram: Development and Operations Architecture



Significant Design Decisions and Rationale

Decision	Rationale	Traceability
Containerization with Kubernetes	Enables consistent deployment across environments.	QA-7, CRN-5
Modular monorepo with shared libraries	Supports independent development while avoiding duplication.	CRN-3, CRN-4
Multi-stage CI/CD pipeline	Ensures quality gates and fast feedback loops.	CRN-5, QA-9
Automated testing with mocks	Isolates services from external dependencies.	QA-9, CRN-4
Environment configuration via IaC	Eliminates manual configuration drift.	QA-7, CRN-5

ADD Step 7: Perform Analysis of Current Design and Review Iteration Goal and Achievement of Design Purpose

Evaluation of Iteration Goal

The iteration goal was to address development and operations concerns by designing a robust CI/CD pipeline, modular architecture, and testing infrastructure. The current design introduces containerization, automated testing, and environment configuration management that directly support these goals. The use of a modular monorepo enables efficient team collaboration and avoids technical debt.

Next Steps

The next iteration would typically involve further refinement of specific components based on feedback or evolving requirements. However, since this is the final iteration outlined in the fixed plan, the overall architecture is now complete and ready for implementation.

STATUS: PASS

Findings: All required ADD steps were completed for Iteration 4. The development and operations structures were refined with CI/CD pipelines, modularization, and testing infrastructure. Design decisions are traceable to use cases, quality attributes, architectural concerns, and constraints. A Mermaid diagram was included to visualize the architecture.

2. Interaction Cost Analysis

Item	Value
The way of completing the assignment	Single-agent, sequential reasoning + self-reflection
The LLM used	Qwen3-32B
Number of Human Interactions (turns)	1 human run command; 8 LLM API calls
Token Consumption (K tokens)	Partially unavailable from API metadata; known total: 0.00
Time Cost (min)	4.57

3. Individual Reflection

Problems Encountered and Solutions Adopted

To be completed by the group after reviewing the generated ADD results and the conversation log.

Personal Contributions

Name (Chinese)	Contributions
王其亮	Built the Spring Boot + Spring AI Alibaba single-agent application, configured the Qwen3-32B DashScope API integration, and fixed runtime issues related to CLI startup, API key loading, and request timeout settings.
廖之乾	Extracted and organized the ADD 3.0 method and Hotel Pricing System prior knowledge from the assignment PDF, then designed the fixed four-iteration ADD workflow and self-reflection mechanism for the single Agent.
谢宪喜	Generated and organized the complete conversation log, ADD outputs, and report draft, including English original outputs and Chinese-readable versions for review and presentation.